
Session 07 (AIML) - Assignment Questions

Name: **Vaishnavi Praveen Rath**

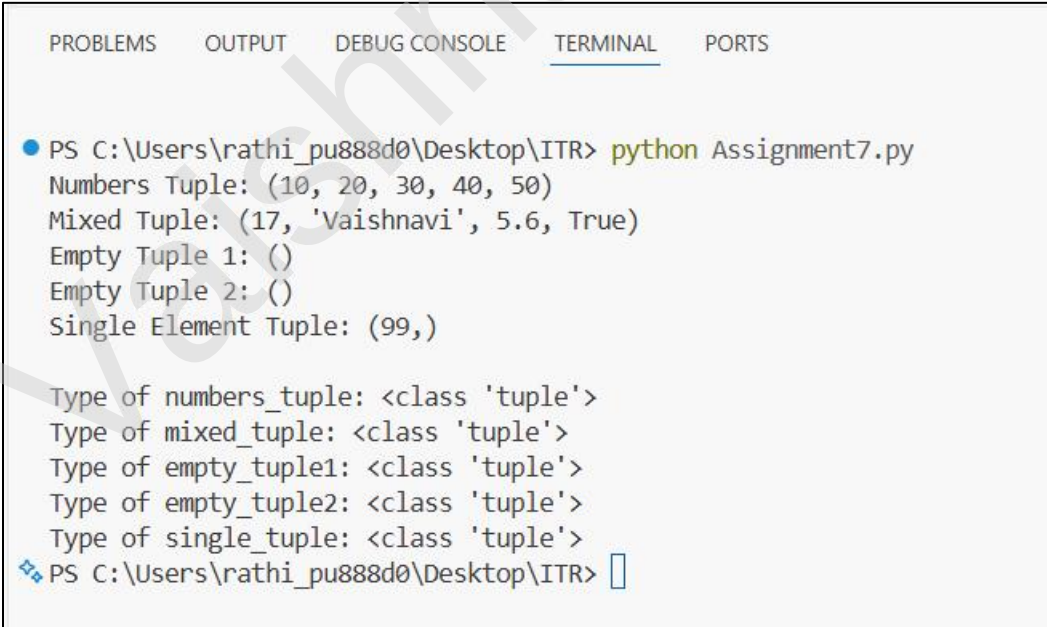
College: **Zeal Polytechnic, Narhe, Pune**

Branch: **Artificial Intelligence and Machine Learning (AIML)**

Domain: **AIML (Artificial Intelligence and Machine Learning)**

Q1. Creating Tuples

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

• PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment7.py
Numbers Tuple: (10, 20, 30, 40, 50)
Mixed Tuple: (17, 'Vaishnavi', 5.6, True)
Empty Tuple 1: ()
Empty Tuple 2: ()
Single Element Tuple: (99,)

Type of numbers_tuple: <class 'tuple'>
Type of mixed_tuple: <class 'tuple'>
Type of empty_tuple1: <class 'tuple'>
Type of empty_tuple2: <class 'tuple'>
Type of single_tuple: <class 'tuple'>
❖ PS C:\Users\rathi_pu888d0\Desktop\ITR> 
```

Program Code:

```
""" ASSIGNMENT 7 """
```

```
'''
```

Q1. (Creating Tuples)

Write a program to create tuples using different methods:

- a) A tuple with 5 numbers using parentheses.
- b) A mixed tuple containing integer, string, float, and boolean.
- c) An empty tuple using two different ways.
- d) A single-element tuple with the value 99.

Print all tuples and their types.

Add comments explaining the rules for single-element tuples.

```
'''
```

```
# a) Tuple with 5 numbers
```

```
numbers_tuple = (10, 20, 30, 40, 50)
```

```
# b) Mixed tuple
```

```
mixed_tuple = (17, "Vaishnavi", 5.6, True)
```

```
# c) Empty tuples using two methods
```

```
empty_tuple1 = ()
```

```
empty_tuple2 = tuple()
```

```
# d) Single-element tuple
```

```
single_tuple = (99,)
```

```
# Print tuples
```

```
print("Numbers Tuple:", numbers_tuple)
```

```
print("Mixed Tuple:", mixed_tuple)
```

```
print("Empty Tuple 1:", empty_tuple1)
```

```
print("Empty Tuple 2:", empty_tuple2)
```

```
print("Single Element Tuple:", single_tuple)
```

```
print()
```

```
# Print types
```

```
print("Type of numbers_tuple:", type(numbers_tuple))
```

```
print("Type of mixed_tuple:", type(mixed_tuple))
```

```
print("Type of empty_tuple1:", type(empty_tuple1))
```

```
print("Type of empty_tuple2:", type(empty_tuple2))
```

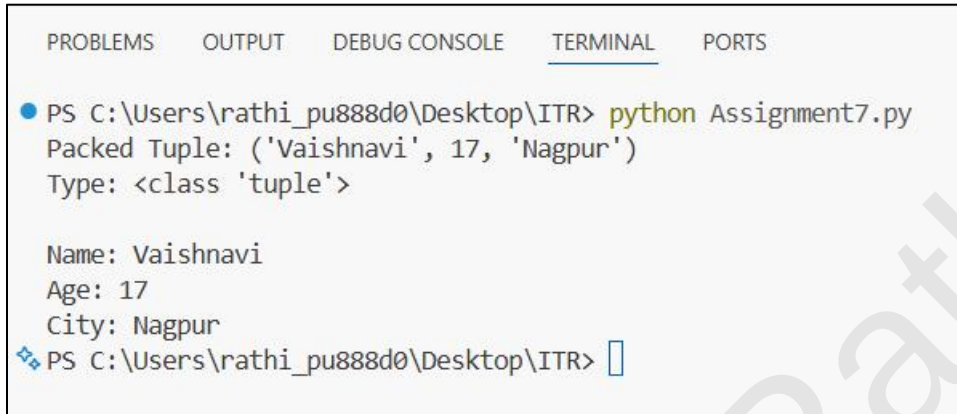
```
print("Type of single_tuple:", type(single_tuple))
```

"Explanation in Comments"

```
# Tuples are created using parentheses ().  
# A tuple can store different data types together.  
# An empty tuple can be created using () or tuple().  
  
# For a single-element tuple, a comma is mandatory.  
# Example: (99,) is a tuple.  
# Example: (99) is treated as an integer, not a tuple.  
  
# Tuples are immutable, which means their elements cannot be changed after creation.
```

Q2. Tuple Packing

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment7.py
Packed Tuple: ('Vaishnavi', 17, 'Nagpur')
Type: <class 'tuple'>

Name: Vaishnavi
Age: 17
City: Nagpur
PS C:\Users\rathi_pu888d0\Desktop\ITR> 
```

Program Code:

```
""" ASSIGNMENT 7 """
```

```
'''
```

```
Q2. (Tuple Packing)
```

```
Create a tuple without using parentheses (tuple packing) to store:
your name, age, and city.
```

```
Print the tuple and its type.
```

```
Then unpack the tuple into three separate variables and print them individually.
```

```
'''
```

```
# Creating a tuple using tuple packing
```

```
student_info = "Vaishnavi", 17, "Nagpur"
```

```
# Print the tuple and its type
```

```
print("Packed Tuple:", student_info)
```

```
print("Type:", type(student_info))
```

```
# Unpacking the tuple into separate variables
```

```
name, age, city = student_info
```

```
# Print individual values
print("\nName:", name)
print("Age:", age)
print("City:", city)
```

"Explanation in Comments"

```
# Tuple packing means creating a tuple without using parentheses.
# Python automatically packs the values into a tuple.
```

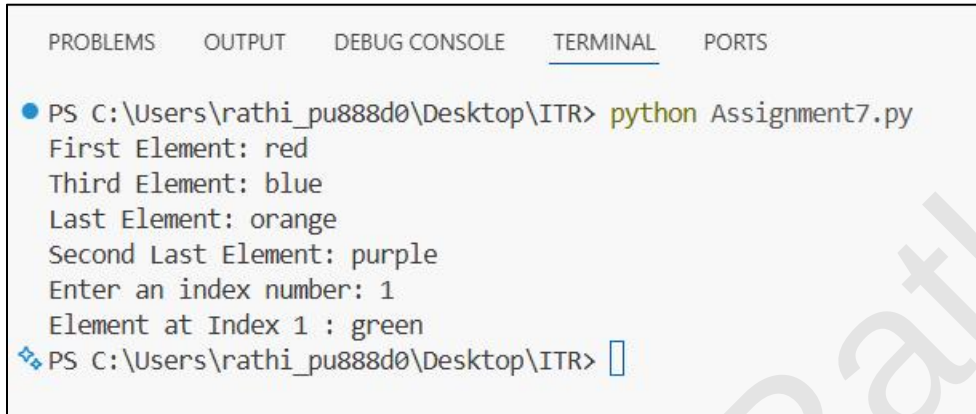
```
# Tuple unpacking means assigning tuple elements to separate variables.
# The number of variables must match the number of elements in the tuple.
```

```
# Packing:
# student_info = "Vaishnavi", 17, "Pune"
```

```
# Unpacking:
# name, age, city = student_info
```

Q3. Indexing and Negative Indexing

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment7.py
First Element: red
Third Element: blue
Last Element: orange
Second Last Element: purple
Enter an index number: 1
Element at Index 1 : green
PS C:\Users\rathi_pu888d0\Desktop\ITR> 
```

Program Code:

```
""" ASSIGNMENT 7 """
```

```
'''
```

Q3. (Indexing and Negative Indexing)

Create the tuple:

```
colors = ('red', 'green', 'blue', 'yellow', 'purple', 'orange')
```

Print:

First element

Third element

Last element (using negative index)

Second last element (using negative index)

Take an index number as input from the user and print the element at that index.

```
'''
```

```
# Creating the tuple
```

```
colors = ('red', 'green', 'blue', 'yellow', 'purple', 'orange')
```

```
# Print required elements
```

```
print("First Element:", colors[0])
```

```
print("Third Element:", colors[2])
```

```
print("Last Element:", colors[-1])
```

```
print("Second Last Element:", colors[-2])
```

```
# Take index input from the user
index = int(input("Enter an index number: "))
```

```
# Print element at the given index
print("Element at Index", index, ":", colors[index])
```

```
"Explanation in Comments"
```

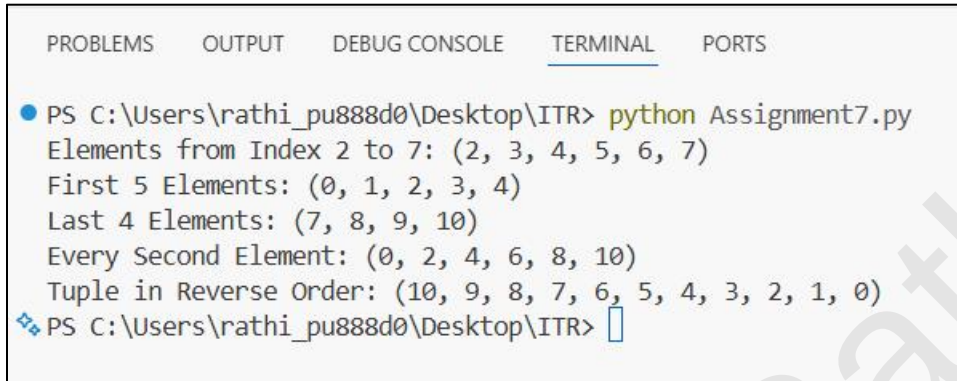
```
# Positive indexing starts from 0 and moves from left to right.
# colors[0] gives the first element.
# colors[2] gives the third element.
```

```
# Negative indexing starts from -1 and moves from right to left.
# colors[-1] gives the last element.
# colors[-2] gives the second last element.
```

```
# Tuples support indexing just like lists.
# The user can enter an index to access a specific element.
.
```

Q4. Slicing Tuples

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment7.py
Elements from Index 2 to 7: (2, 3, 4, 5, 6, 7)
First 5 Elements: (0, 1, 2, 3, 4)
Last 4 Elements: (7, 8, 9, 10)
Every Second Element: (0, 2, 4, 6, 8, 10)
Tuple in Reverse Order: (10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0)
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

Program Code:

```
""" ASSIGNMENT 7 """
```

```
'''
```

Q4. (Slicing Tuples)

Given the tuple:

```
numbers = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

Using slicing, print:

- a) Elements from index 2 to 7
- b) First 5 elements
- c) Last 4 elements
- d) Every second element
- e) The entire tuple in reverse order

Add comments explaining the slicing syntax.

```
'''
```

```
# Creating the tuple
```

```
numbers = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

```
# a) Elements from index 2 to 7
```

```
print("Elements from Index 2 to 7:", numbers[2:8])
```

```
# b) First 5 elements
```

```
print("First 5 Elements:", numbers[:5])
```



```
# c) Last 4 elements
print("Last 4 Elements:", numbers[-4:])

# d) Every second element
print("Every Second Element:", numbers[::2])

# e) Entire tuple in reverse order
print("Tuple in Reverse Order:", numbers[::-1])
```

"Explanation in Comments"

Tuple slicing uses the syntax [start:end:step].

start -> Starting index (inclusive).

end -> Ending index (exclusive).

step -> Number of positions to move.

numbers[2:8] returns elements from index 2 to 7.

numbers[:5] returns the first 5 elements.

numbers[-4:] returns the last 4 elements.

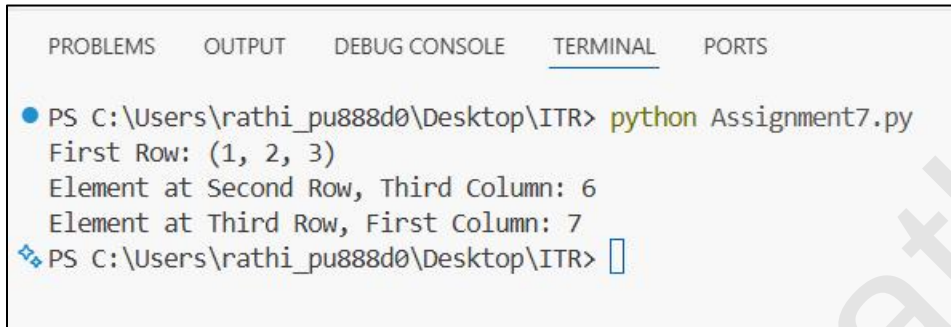
numbers[::2] returns every second element.

numbers[::-1] reverses the tuple.

Slicing allows us to access a portion of a tuple easily.

Q5. Nested Tuples

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment7.py
First Row: (1, 2, 3)
Element at Second Row, Third Column: 6
Element at Third Row, First Column: 7
PS C:\Users\rathi_pu888d0\Desktop\ITR> 
```

Program Code:

```
""" ASSIGNMENT 7 """
```

```
'''
```

Q5. (Nested Tuples)

Create a nested tuple to represent a 3x3 matrix:

```
matrix = ((1, 2, 3), (4, 5, 6), (7, 8, 9))
```

Print:

The first row

The element at second row, third column

The element at third row, first column

```
'''
```

```
# Creating a nested tuple (3x3 matrix)
```

```
matrix = ((1, 2, 3),
          (4, 5, 6),
          (7, 8, 9))
```

```
# Print the first row
```

```
print("First Row:", matrix[0])
```

```
# Print the element at second row, third column
```

```
print("Element at Second Row, Third Column:", matrix[1][2])
```

```
# Print the element at third row, first column
```

```
print("Element at Third Row, First Column:", matrix[2][0])
```

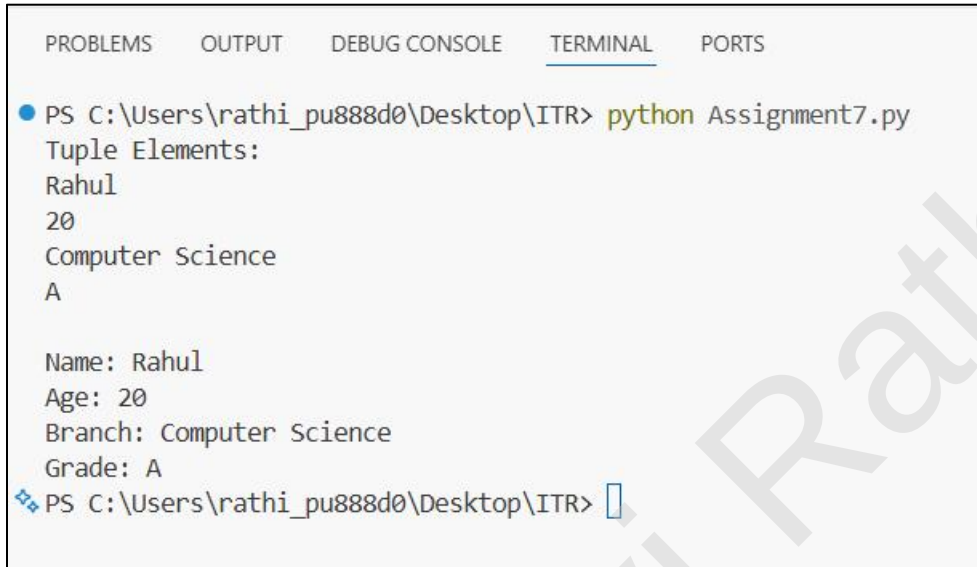
"Explanation in Comments"

```
# A nested tuple is a tuple that contains other tuples as elements.  
# matrix[0] accesses the first row.  
# matrix[1][2] accesses the second row and third column.  
# matrix[2][0] accesses the third row and first column.  
  
# The first index selects the row.  
# The second index selects the column within that row.
```

Vaishnavi Rathni

Q6. Iterating & Unpacking

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\rathi_pu88d0\Desktop\ITR> python Assignment7.py
Tuple Elements:
Rahul
20
Computer Science
A

Name: Rahul
Age: 20
Branch: Computer Science
Grade: A
PS C:\Users\rathi_pu88d0\Desktop\ITR> 
```

Program Code:

```
""" ASSIGNMENT 7 """
```

```
'''
```

Q6. (Iterating and Unpacking)

Create a tuple:

```
student = ('Rahul', 20, 'Computer Science', 'A')
```

a) Iterate through the tuple using a for loop and print each item.

b) Unpack the tuple into four variables (name, age, branch, grade) and print them with descriptive messages.

```
'''
```

```
# Creating the tuple
```

```
student = ('Rahul', 20, 'Computer Science', 'A')
```

```
# a) Iterate through the tuple using a for loop
```

```
print("Tuple Elements:")
```

```
for item in student:
```

```
    print(item)
```

```
print()
```

```
# b) Unpack the tuple into variables
name, age, branch, grade = student
```

```
# Print the unpacked values
```

```
print("Name:", name)
print("Age:", age)
print("Branch:", branch)
print("Grade:", grade)
```

```
"""Explanation in Comments"""
```

```
# A for loop can be used to access each element of a tuple one by one.
```

```
# The variable 'item' stores each element during iteration.
```

```
# Tuple unpacking assigns tuple elements to separate variables.
```

```
# The number of variables must match the number of tuple elements.
```

```
# name receives 'Rahul'
```

```
# age receives 20
```

```
# branch receives 'Computer Science'
```

```
# grade receives 'A'
```

Q7. count() Method

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment7.py
Count of A: 4
Count of B: 3
Enter a grade: C
Count of C : 1
PS C:\Users\rathi_pu888d0\Desktop\ITR> 
```

Program Code:

```
""" ASSIGNMENT 7 """
```

```
'''
```

```
Q7. (count() Method)
```

```
Create the tuple:
```

```
grades = ('A', 'B', 'A', 'C', 'A', 'B', 'D', 'A', 'B')
```

```
Count and print how many times 'A' appears.
```

```
Count and print how many times 'B' appears.
```

```
Take a grade as input from the user and print how many times it appears.
```

```
'''
```

```
# Creating the tuple
```

```
grades = ('A', 'B', 'A', 'C', 'A', 'B', 'D', 'A', 'B')
```

```
# Count occurrences of A and B
```

```
print("Count of A:", grades.count('A'))
```

```
print("Count of B:", grades.count('B'))
```

```
# Take a grade as input from the user
```

```
grade = input("Enter a grade: ")
```

```
# Print the count of the entered grade
```

```
print("Count of", grade, ":", grades.count(grade))
```

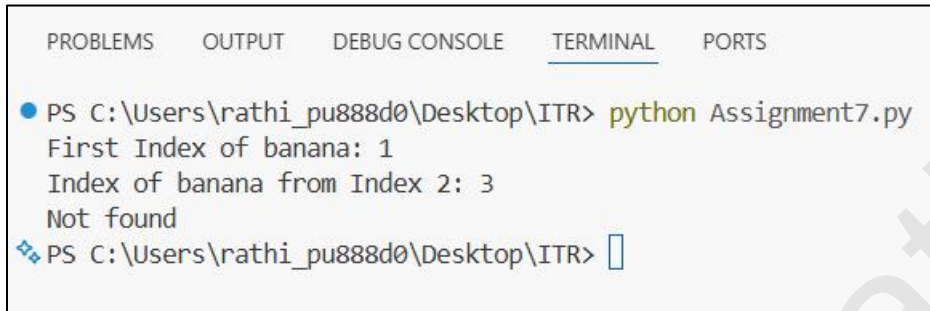
"Explanation in Comments"

```
# count() is used to find how many times a value appears in a tuple.  
# grades.count('A') returns the number of times 'A' occurs.  
# grades.count('B') returns the number of times 'B' occurs.  
# The user can enter any grade, and the program will display its count.  
# If the grade is not present, count() returns 0.
```

Vaishnavi Rathni

Q8. index() Method

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\rathi_pu88d0\Desktop\ITR> python Assignment7.py
First Index of banana: 1
Index of banana from Index 2: 3
Not found
❖ PS C:\Users\rathi_pu88d0\Desktop\ITR> 
```

Program Code:

```
""" ASSIGNMENT 7 """
```

```
'''
```

Q8. (index() Method)

Create the tuple:

```
fruits = ('apple', 'banana', 'cherry', 'banana', 'mango', 'apple')
```

Find and print the first index of 'banana'.

Find and print the first index of 'banana' starting the search from index 2.

Use try-except to safely find the index of 'kiwi' and print "Not found" if it doesn't exist.

```
'''
```

```
# Creating the tuple
```

```
fruits = ('apple', 'banana', 'cherry', 'banana', 'mango', 'apple')
```

```
# Find the first index of 'banana'
```

```
print("First Index of banana:", fruits.index('banana'))
```

```
# Find the first index of 'banana' starting from index 2
```

```
print("Index of banana from Index 2:", fruits.index('banana', 2))
```

```
# Use try-except to find 'kiwi' safely
```

```
try:
```

```
    print("Index of kiwi:", fruits.index('kiwi'))
```

```
except ValueError:
```

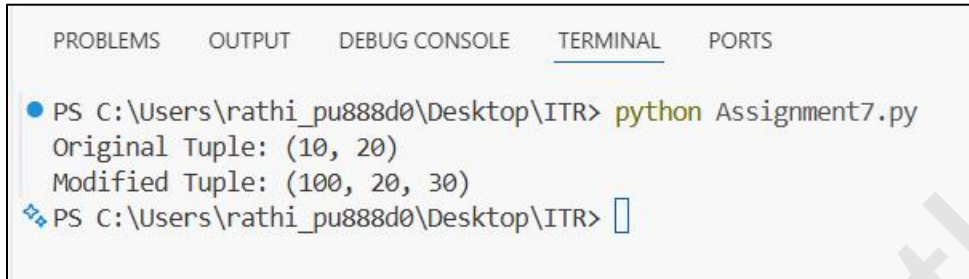
```
    print("Not found")
```


"Explanation in Comments"

```
# index(value) returns the first occurrence of the specified value.  
# fruits.index('banana') returns the first index where 'banana' is found.  
  
# index(value, start) starts searching from the given index.  
# fruits.index('banana', 2) searches for 'banana' beginning at index 2.  
  
# If the value does not exist, index() raises a ValueError.  
# try-except is used to handle this error and prevent the program from crashing.  
# In this example, 'kiwi' is not present, so "Not found" is printed.
```

Q9. Immutability of Tuples

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment7.py
Original Tuple: (10, 20)
Modified Tuple: (100, 20, 30)
PS C:\Users\rathi_pu888d0\Desktop\ITR> 
```

Program Code:

```
""" ASSIGNMENT 7 """
```

```
'''
```

Q9. (Immutability of Tuples)

Create a tuple:

```
coordinates = (10, 20)
```

Demonstrate that tuples are immutable by trying to:

Change the first element

Add a new element using append()

Explain what errors occur (in comments).

Then show the correct way to modify data by converting the tuple to a list, making changes, and converting back to a tuple.

```
'''
```

```
# Creating the tuple
```

```
coordinates = (10, 20)
```

```
print("Original Tuple:", coordinates)
```

```
# The following statement causes an error because tuples are immutable
```

```
# coordinates[0] = 100
```

```
# Error:
```

```
# TypeError: 'tuple' object does not support item assignment
```

```
# The following statement causes an error because tuples do not have append()
```

```
# coordinates.append(30)

# Error:
# AttributeError: 'tuple' object has no attribute 'append'

# Correct way: Convert tuple to list
coordinates_list = list(coordinates)

# Modify the first element
coordinates_list[0] = 100

# Add a new element
coordinates_list.append(30)

# Convert the list back to a tuple
coordinates = tuple(coordinates_list)

print("Modified Tuple:", coordinates)

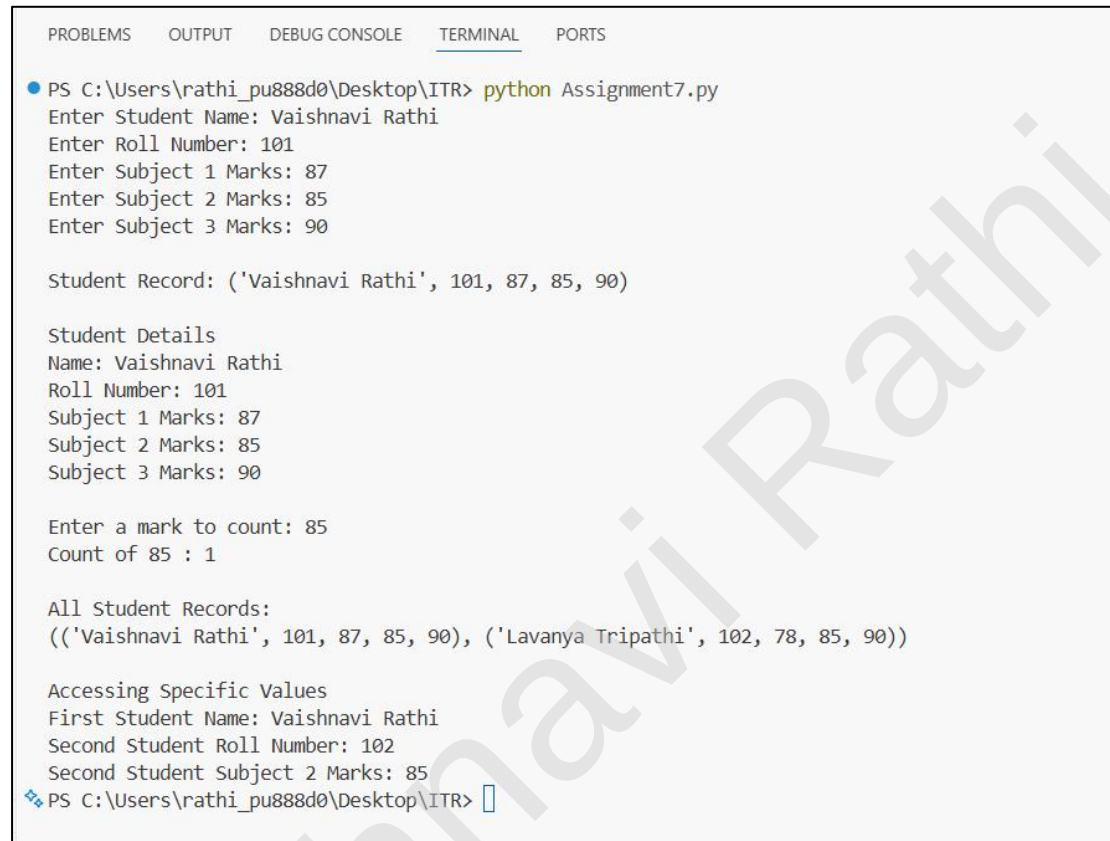
"""Explanation in Comments"""

# Tuples are immutable, which means their elements cannot be changed.
# We cannot assign new values to tuple elements.
# Tuples also do not support methods like append(), remove(), or insert().

# To modify tuple data:
# 1. Convert the tuple into a list using list().
# 2. Make the required changes in the list.
# 3. Convert the list back into a tuple using tuple().
```

Q10. Mini Project - Combined Concepts

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment7.py
Enter Student Name: Vaishnavi Rathi
Enter Roll Number: 101
Enter Subject 1 Marks: 87
Enter Subject 2 Marks: 85
Enter Subject 3 Marks: 90

Student Record: ('Vaishnavi Rathi', 101, 87, 85, 90)

Student Details
Name: Vaishnavi Rathi
Roll Number: 101
Subject 1 Marks: 87
Subject 2 Marks: 85
Subject 3 Marks: 90

Enter a mark to count: 85
Count of 85 : 1

All Student Records:
(('Vaishnavi Rathi', 101, 87, 85, 90), ('Lavanya Tripathi', 102, 78, 85, 90))

Accessing Specific Values
First Student Name: Vaishnavi Rathi
Second Student Roll Number: 102
Second Student Subject 2 Marks: 85
PS C:\Users\rathi_pu888d0\Desktop\ITR> 
```

Program Code:

```
""" ASSIGNMENT 7 """
```

Q10. (Mini Project - Combined Concepts)

Create a Student Record Program using tuples:

Take input for name, roll number, 3 subject marks.

Store all information in a tuple (use packing).

Print the complete record.

Unpack the tuple and print each value with proper labels.

Use count() to check how many times a particular mark appears (if any).

Create a nested tuple to store multiple student records (at least 2 students) and access specific values.

Add meaningful comments and use proper formatting.
'''

```
# Take input from the user
name = input("Enter Student Name: ")
roll_number = int(input("Enter Roll Number: "))
mark1 = int(input("Enter Subject 1 Marks: "))
mark2 = int(input("Enter Subject 2 Marks: "))
mark3 = int(input("Enter Subject 3 Marks: "))

# Create a tuple using packing
student_record = name, roll_number, mark1, mark2, mark3

# Print the complete record
print("\nStudent Record:", student_record)

# Unpack the tuple into separate variables
student_name, student_roll, subject1, subject2, subject3 = student_record

# Print unpacked values with labels
print("\nStudent Details")
print("Name:", student_name)
print("Roll Number:", student_roll)
print("Subject 1 Marks:", subject1)
print("Subject 2 Marks:", subject2)
print("Subject 3 Marks:", subject3)

# Take a mark from the user and count its occurrences
search_mark = int(input("\nEnter a mark to count: "))
print("Count of", search_mark, ":", student_record.count(search_mark))

# Create a nested tuple for multiple student records
student_records = (
    student_record,
    ("Lavanya Tripathi", 102, 78, 85, 90)
)

# Print all student records
print("\nAll Student Records:")
print(student_records)

# Access specific values from the nested tuple
print("\nAccessing Specific Values")
print("First Student Name:", student_records[0][0])
print("Second Student Roll Number:", student_records[1][1])
print("Second Student Subject 2 Marks:", student_records[1][3])
```

"Explanation in Comments"

```
# Tuple packing allows multiple values to be stored together.  
# Tuple unpacking separates tuple elements into individual variables.  
# count() returns how many times a value appears in a tuple.  
# A nested tuple is a tuple containing other tuples.  
# student_records stores information of multiple students.  
# student_records[row][column] is used to access specific values.  
# Tuples are immutable, so their elements cannot be modified directly.
```

Vaishnavi Rathin